

PROJECT BLUEPRINT BIBLE

AI Product Video Ad Kitchen / Product Robot Video Factory

0. Project Codename

AI Product Video Ad Kitchen

หรืออีกชื่อหนึ่ง:

Product Robot Video Factory

ระบบนี้คือระบบผลิตคลิปโฆษณาสินค้าแบบกึ่งอัตโนมัติ โดยใช้แนวคิดว่า

สินค้า 1 ตัว = Product Robot 1 ตัว

Product Robot แต่ละตัวมีคลังวัตถุดิบของตัวเอง เช่น foreground video, background video, voiceover, music, script, template, tag และ recipe

ระบบจะนำวัตถุดิบเหล่านี้มาประกอบเป็นคลิปโฆษณาหลายแบบ โดยมี workflow, quality gate, performance optimization, cache และ orchestrator ควบคุมการทำงาน

1. Core Vision

ระบบนี้ไม่ใช่แค่โปรแกรมตัดต่อวิดีโอ แต่เป็น โรงงานผลิตคลิปโฆษณาแบบมีสมอง

เป้าหมายคือให้ผู้ใช้สามารถถ่ายคลิปนำเสนอสินค้าบน green screen แล้วนำมาซ้อนกับ background video หลายแบบ พร้อมเสียงพากย์ เพลง ข้อความ CTA และ template เพื่อสร้างคลิปโฆษณาจำนวนมาก แต่ยังคงคุณภาพ ความแตกต่าง และความเหมาะสมทางการตลาดได้

แนวคิดหลัก:

Foreground Video = คลิปนำเสนอสินค้า ถ่ายบน Green Screen
Background Video = ฉากหลัง เช่น ตลาด คาเฟ่ คาร์ สตูดิโอ ธรรมชาติ
Voiceover = เสียงพูดนำเสนอสินค้า
Music = เพลงประกอบ
Template = layout, caption, logo, price banner, CTA
Recipe = สูตรการประกอบวิดีโอหนึ่งคลิป
Orchestrator = ตัวควบคุม workflow ทั้งหมด

Worker = ตัวทำงานย่อย เช่น analyze, proxy, alpha, render, audio, quality check
SQLite = ฐานข้อมูลความจำของระบบ
Media Folder = คลังไฟล์วิดีโอ/เสียงจริง

2. Main Product Concept

สินค้าแต่ละตัวมี Product Robot ของตัวเอง

ตัวอย่าง:

```
products/  
└─ biiigbee_honey/  
    │─ background_videos/  
    │─ foreground_videos/  
    │─ voiceovers/  
    │─ background_music/  
    │─ scripts/  
    │─ templates/  
    │─ outputs/  
    └─ cache/
```

Product Robot ของสินค้านี้จะรู้ไว้:

สินค้านี้คืออะไร
การใช้ background แบบไหน
การใช้เสียงพากย์แบบไหน
การใช้เพลง mood อะไร
การใช้ hook / CTA แบบไหน
การสร้างคลิปสำหรับ platform อะไร
การ reject คลิปแบบไหน

3. Primary Use Case

ผู้ใช้งานต้องการสร้างคลิปโฆษณาสินค้าจำนวนมากจากวัตถุดิบที่เตรียมไว้

ตัวอย่าง flow:

1. ผู้ใช้สร้าง Product Robot ชื่อ BIIIGBEE Honey
2. ผู้ใช้อัปโหลด foreground video ที่ถ่ายบน green screen
3. ผู้ใช้อัปโหลด background videos หลายแบบ
4. ผู้ใช้อัปโหลด voiceover หลายไฟล์
5. ผู้ใช้อัปโหลด background music หลายไฟล์
6. ผู้ใช้ใส่ tag ให้แต่ละ asset ผ่าน UI
7. ระบบสร้าง recipe หลายชุด
8. ระบบ render preview
9. ระบบให้คะแนน quality / duplicate / fit
10. ผู้ใช้เลือก approve
11. ระบบ render final output
12. ระบบบันทึก output พร้อม report

4. System Architecture

4.1 High-Level Architecture

```
UI / Dashboard
  ↓
Application Service
  ↓
Orchestrator
  ↓
Job Queue
  ↓
Workers
  ↓
Media Storage + Cache + SQLite
```

4.2 Recommended Version 1 Stack

```
Language: Python
UI: Streamlit
Database: SQLite
Video Processing: FFmpeg + OpenCV
Storage: Local Folder
Worker: Python multiprocessing / threading
Config: YAML / JSON
```

4.3 Future Production Stack

Frontend: React
Backend: FastAPI
Database: PostgreSQL
Queue: Redis + RQ/Celery
Storage: MinIO / S3 compatible storage
GPU Worker: Dedicated render server
Monitoring: Prometheus / Grafana

5. Core Modules

5.1 Media Library Manager

หน้าที่:

จัดการไฟล์สื่อทั้งหมด
อัปโหลดไฟล์
อ่าน metadata
สร้าง thumbnail
สร้าง proxy
ใส่ tag
ค้นหา asset
จัดกลุ่มตาม product

Asset types:

background_video
foreground_video
voiceover
background_music
sfx
template
script
hook
cta

5.2 Tag-Based Asset System

ระบบต้องใช้ tag เป็นหัวใจของการเลือกวัตถุดิบ

Tag groups:

```
asset_type
product
mood
scene
psychology
platform
layout
motion
voice_type
foreground_action
energy
script_angle
```

ตัวอย่าง tag:

```
mood: warm, premium, natural, energetic, cozy, trustworthy
scene: market, cafe, kitchen, studio, nature, warehouse, gift_table
psychology: social_proof, urgency, trust_building, price_anchor, demo_style
platform: tiktok, facebook_reels, youtube_shorts, shopee_video
layout: left_safe, right_safe, center_empty, caption_bottom, price_top
motion: low_motion, medium_motion, fast_motion, loopable
foreground_action: hold_product, point_left, show_label, hand_demo, smile_cta
```

หลักการ:

```
Folder = จัดระเบียบไฟล์
Filename = อธิบายแบบสั้น
Tag = อธิบายเชิงลึกเพื่อให้ระบบเลือก asset ได้ฉลาด
SQLite = เก็บข้อมูลการ
```

5.3 Product Robot

Product Robot คือ logical unit ของสินค้าหนึ่งตัว

หน้าที่:

```
อ่าน profile ของสินค้า
อ่าน asset ทั้งหมดของสินค้า
เลือก asset ตาม tag
สร้าง recipe
ตรวจสอบความเหมาะสม
ส่ง recipe ไป preview
ส่ง recipe ที่ผ่านไป final render
เก็บผลลัพธ์และ report
```

Product Robot ไม่ควร render เอง
Product Robot ควรสร้างคำสั่งให้ Orchestrator

5.4 Recipe Engine

Recipe คือสูตรการประกอบคลิปหนึ่งคลิป

Recipe ประกอบด้วย:

```
foreground video
background video
voiceover
background music
template
hook
caption
CTA
platform
duration
ratio
composition settings
audio settings
```

ตัวอย่าง recipe:

```
recipe_id: honey_recipe_001
product: biiigbee_honey
foreground: fg_honey_presenter_holdproduct_001
background: bg_honey_morningcafe_001
voiceover: voice_honey_femalewarm_morningdrink_001
music: music_honey_acoustic_warm_001
template: tpl_caption_bottom_price_001
hook: "กาแฟแก้วเดิม อร่อยขึ้นได้ง่าย ๆ"
```

```
cta: "กดดูสูตรเพิ่มเติ่ม"  
platform: tiktok  
ratio: 9:16  
duration_sec: 10  
mood: warm_natural
```

Recipe Engine ต้องไม่สับสนว่า แต่ควรใช้:

```
Random candidate generation  
Rule-based filtering  
Tag matching  
Scoring  
Quality gate  
Duplicate/repetition check
```

5.5 Green Screen Compositor

หน้าที่:

```
รับ foreground video ที่ถ่ายบน green screen  
ตัดสีเขียวออก  
สร้าง alpha matte  
ลบ green spill  
refine edge  
นำ foreground ไปซ้อนกับ background video  
ปรับสี แสง เงา และความกลืน
```

Workflow:

```
Foreground Green Screen Video  
↓  
Chroma Key  
↓  
Alpha Matte  
↓  
Spill Removal  
↓  
Edge Feather  
↓  
Foreground RGBA Cache
```

↓
Composite with Background

หลัก performance สำคัญ:

Key foreground once, reuse many times

ห้ามทำ chroma key ซ้ำทุก recipe ถ้า foreground เดิม

5.6 Audio Engine

หน้าที่:

ผสม voiceover + background music + sfx
normalize voice
duck music ได้เสียงพูด
loop music ถ้าสั้น
fade in / fade out
target loudness
export audio track

คำแนะนำ:

Voice must be clear
Music must not overpower voice
Use music ducking when voice is present
Target loudness for social video: approximately -14 LUFS

5.7 Template / Overlay Engine

Layer structure:

Layer 0: Background Video
Layer 1: Gradient / Vignette / Blur Overlay
Layer 2: Foreground Presenter/Product
Layer 3: Shadow / Rim Light
Layer 4: Price Banner
Layer 5: Subtitle

Layer 6: Logo
Layer 7: CTA Button / Message Prompt

Template presets:

```
clean_caption_bottom  
price_banner_top  
market_promo_big_offer  
premium_minimal  
review_style  
live_commerce_style  
product_demo_style
```

5.8 Quality Gate Engine

ตรวจสอบคุณภาพก่อน final render

Checks:

```
product visibility  
face visibility  
foreground edge quality  
green spill risk  
audio clarity  
music/voice balance  
background distraction  
text safe area  
CTA clarity  
platform ratio correctness  
duplicate/repetition risk  
render error  
file playable
```

Decision:

```
APPROVE  
REVISE  
REJECT  
NEEDS_HUMAN_REVIEW
```

5.9 Orchestrator

Orchestrator คือสมองควบคุม workflow

หน้าที่:

```
รับคำสั่งจาก UI
สร้าง job
แตก job เป็น step
จัดลำดับงาน
ส่งงานให้ worker
ควบคุมจำนวน worker
ตรวจ resource
pause / resume / cancel
retry เมื่อ failed
บันทึกสถานะลง SQLite
```

Orchestrator ไม่ควร render เอง

Orchestrator ควบคุม worker

6. Worker Architecture

Workers ที่ควรมี:

```
AssetAnalyzerWorker
ProxyWorker
AlphaWorker
PreviewRenderWorker
FinalRenderWorker
AudioWorker
QualityCheckWorker
DuplicateCheckWorker
```

6.1 AssetAnalyzerWorker

ทำงานเมื่อ upload asset

```
อ่าน duration
fps
resolution
ratio
```

```
codec
has_audio
file size
สร้าง thumbnail
บันทึกลง SQLite
```

6.2 ProxyWorker

สร้าง proxy video สำหรับ preview

```
Original 4K → proxy 540x960 หรือ 720x1280
```

Preview ใช้ proxy

Final ใช้ original

6.3 AlphaWorker

ทำกับ foreground green screen

```
foreground green screen
→ alpha matte
→ foreground RGBA cache
```

6.4 PreviewRenderWorker

render preview คุณภาพต่ำ

```
recipe → preview mp4
```

ใช้สำหรับ review ก่อน final

6.5 FinalRenderWorker

render final output คุณภาพสูง

```
approved recipe → final mp4
```

ควรจำกัดจำนวน worker น้อยๆ ระวัง resource สูง

6.6 AudioWorker

```
normalize voice  
duck music  
mix voice + music  
export final audio
```

6.7 QualityCheckWorker

```
ตรวจคุณภาพ output  
ตรวจ playable  
ตรวจเสียง  
ตรวจความคม  
ตรวจ duplicate risk
```

7. Resource Management

ระบบควรมี Resource Manager เพื่อลดความเสี่ยงเครื่องค้าง

Monitor:

```
CPU usage  
RAM usage  
GPU usage  
disk free space  
active render jobs  
temperature if possible
```

Rules:

```
ถ้า RAM > 80% ห้ามเริ่ม final render ใหม่  
ถ้า disk free < 20GB pause queue  
ถ้ามี final render อยู่แล้วไม่เริ่ม final render ตัวที่สอง  
ถ้า CPU > 90% ต่อเนื่อง ลดจำนวน preview worker
```

Recommended default worker count:

```
workers:
  analyzer: 2
  proxy: 1
  alpha: 1
  preview_render: 1
  final_render: 1
  audio: 1
  quality_check: 1
```

8. Workflow Process

8.1 Full Workflow

```
Create Product
  ↓
Upload Assets
  ↓
Analyze Assets
  ↓
Create Thumbnail / Proxy
  ↓
Manual + Auto Tagging
  ↓
Generate Recipes
  ↓
Filter by Rules
  ↓
Score Recipes
  ↓
Render Preview
  ↓
Quality Check
  ↓
Human Review
  ↓
Render Final
  ↓
Save Output + Report
```

8.2 Preview-First Principle

ห้าม render final ทุก candidate

ควรทำแบบนี้:

```
generate 300 candidates
→ filter เหลือ 100
→ preview 100
→ human approve 20
→ final render 20
```

9. Performance Principles

หลักสำคัญ:

1. Analyze asset once
2. Create proxy once
3. Create alpha matte once
4. Cache reusable intermediate files
5. Preview before final
6. Do not render rejected recipe
7. Use queue
8. Use resource limit
9. Store every state in SQLite
10. Resume after crash

Cache types:

```
metadata cache
thumbnail cache
proxy video cache
alpha matte cache
foreground RGBA cache
normalized audio cache
preview cache
quality score cache
```

10. SQLite Database Design

10.1 products

```
CREATE TABLE products (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  product_code TEXT UNIQUE NOT NULL,  
  product_name TEXT NOT NULL,  
  category TEXT,  
  brand_name TEXT,  
  description TEXT,  
  default_platform TEXT,  
  created_at TEXT DEFAULT CURRENT_TIMESTAMP  
);
```

10.2 assets

```
CREATE TABLE assets (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  product_id INTEGER,  
  asset_code TEXT UNIQUE NOT NULL,  
  asset_type TEXT NOT NULL,  
  file_path TEXT NOT NULL,  
  file_name TEXT NOT NULL,  
  
  duration_sec REAL,  
  width INTEGER,  
  height INTEGER,  
  fps REAL,  
  ratio TEXT,  
  file_size_mb REAL,  
  codec TEXT,  
  has_audio INTEGER DEFAULT 0,  
  
  thumbnail_path TEXT,  
  proxy_path TEXT,  
  alpha_path TEXT,  
  rgba_cache_path TEXT,  
  
  quality_score REAL DEFAULT 0,  
  status TEXT DEFAULT 'active',  
  
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
```

```
FOREIGN KEY (product_id) REFERENCES products(id)
);
```

10.3 tags

```
CREATE TABLE tags (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  tag_name TEXT NOT NULL,
  tag_group TEXT NOT NULL,
  description TEXT,
  UNIQUE(tag_name, tag_group)
);
```

10.4 asset_tags

```
CREATE TABLE asset_tags (
  asset_id INTEGER NOT NULL,
  tag_id INTEGER NOT NULL,
  PRIMARY KEY (asset_id, tag_id),
  FOREIGN KEY (asset_id) REFERENCES assets(id),
  FOREIGN KEY (tag_id) REFERENCES tags(id)
);
```

10.5 recipes

```
CREATE TABLE recipes (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  product_id INTEGER NOT NULL,
  recipe_code TEXT UNIQUE NOT NULL,

  target_platform TEXT,
  target_ratio TEXT,
  duration_sec REAL,

  mood TEXT,
  script_angle TEXT,
  target_audience TEXT,
  hook_text TEXT,
  cta_text TEXT,

  recipe_score REAL DEFAULT 0,
  duplicate_risk REAL DEFAULT 0,
  status TEXT DEFAULT 'candidate',
```



```
created_at TEXT DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (product_id) REFERENCES products(id)  
);
```

10.6 recipe_items

```
CREATE TABLE recipe_items (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  recipe_id INTEGER NOT NULL,  
  asset_id INTEGER NOT NULL,  
  role TEXT NOT NULL,  
  
  FOREIGN KEY (recipe_id) REFERENCES recipes(id),  
  FOREIGN KEY (asset_id) REFERENCES assets(id)  
);
```

Roles:

```
foreground  
background  
voiceover  
music  
template  
sfx
```

10.7 jobs

```
CREATE TABLE jobs (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  job_code TEXT UNIQUE NOT NULL,  
  job_type TEXT NOT NULL,  
  
  recipe_id INTEGER,  
  asset_id INTEGER,  
  
  status TEXT DEFAULT 'pending',  
  priority INTEGER DEFAULT 5,  
  
  progress REAL DEFAULT 0,  
  worker_id TEXT,
```

```
input_json TEXT,  
output_json TEXT,  
  
error_message TEXT,  
  
created_at TEXT DEFAULT CURRENT_TIMESTAMP,  
started_at TEXT,  
finished_at TEXT  
);
```

Job types:

```
analyze_asset  
create_proxy  
create_alpha  
render_preview  
render_final  
mix_audio  
quality_check  
duplicate_check
```

Status:

```
pending  
queued  
processing  
paused  
done  
failed  
cancelled
```

10.8 outputs

```
CREATE TABLE outputs (  
  id INTEGER PRIMARY KEY AUTOINCREMENT,  
  recipe_id INTEGER NOT NULL,  
  
  output_code TEXT UNIQUE NOT NULL,  
  file_path TEXT NOT NULL,  
  platform TEXT,  
  ratio TEXT,  
  duration_sec REAL,
```

```
quality_score REAL,  
duplicate_risk REAL,  
approved INTEGER DEFAULT 0,  
  
created_at TEXT DEFAULT CURRENT_TIMESTAMP,  
  
FOREIGN KEY (recipe_id) REFERENCES recipes(id)  
);
```

11. Folder Structure

```
project_root/  
├── app/  
│   ├── ui/  
│   │   └── streamlit_app.py  
│   ├── orchestrator/  
│   │   ├── orchestrator.py  
│   │   ├── scheduler.py  
│   │   ├── resource_manager.py  
│   │   └── job_repository.py  
│   ├── workers/  
│   │   ├── analyze_worker.py  
│   │   ├── proxy_worker.py  
│   │   ├── alpha_worker.py  
│   │   ├── preview_worker.py  
│   │   ├── final_render_worker.py  
│   │   ├── audio_worker.py  
│   │   └── quality_worker.py  
│   ├── media_library/  
│   ├── recipe_engine/  
│   ├── compositor/  
│   ├── audio/  
│   ├── database/  
│   └── utils/  
├── media_library/  
│   └── products/  
│       ├── biiigbee_honey/  
│       │   ├── background_videos/  
│       │   ├── foreground_videos/  
│       │   ├── voiceovers/  
│       │   ├── background_music/  
│       │   └── templates/
```


Voiceover

```
voice_[product]_[scriptangle]_[voicetype]_[emotion]_[duration]_[version].wav
```

Example:

```
voice_honey_morningdrink_femalewarm_trust_10s_v001.wav
```

Music

```
music_[product]_[style]_[mood]_[energy]_[loop]_[version].mp3
```

Example:

```
music_honey_acoustic_warm_low_loop30s_v001.mp3
```

หมายเหตุ:

```
ชื่อไฟล์ = metadata แบบสั้น  
SQLite/tag = metadata แบบละเอียด
```

13. UI Requirements

13.1 Product Dashboard

ต้องแสดงสินค้าแต่ละตัว:

```
Product name  
Asset count  
Background count  
Foreground count  
Voice count  
Music count  
Recipe count  
Output count
```

13.2 Media Library Page

Tabs:

```
Background Videos
Foreground Videos
Voiceovers
Background Music
Templates
Scripts / Hooks / CTA
```

Each asset card:

```
thumbnail / preview
asset name
duration
ratio
tags
quality score
buttons: preview, edit tags, use in recipe
```

13.3 Upload Page

User selects:

```
asset type
product
file
tags
notes
```

System automatically:

```
copies file to correct folder
renames if needed
analyzes metadata
creates thumbnail
creates DB record
```

13.4 Tag Editor

ใช้ dropdown / checkbox / tag chips

ไม่ควรปล่อยให้ user พิมพ์ tag อิสระโดยไม่มี dictionary

13.5 Recipe Builder

Modes:

```
Manual Recipe
Auto Generate Recipes
Generate Top N Recipes
```

13.6 Preview Review Page

Show:

```
preview video
recipe components
recipe score
quality score
duplicate risk
buttons: approve, reject, revise, render final
```

13.7 Job Monitor

Show:

```
pending jobs
processing jobs
done jobs
failed jobs
worker status
CPU/RAM/disk usage
pause/resume/cancel buttons
```

14. Orchestrator State Machine

Recipe state:

```
CREATED
ASSETS_READY
ALPHA_READY
PREVIEW_PENDING
PREVIEW_RENDERING
PREVIEW_RENDERED
QUALITY_CHECK_PENDING
QUALITY_CHECKED
NEEDS_HUMAN_REVIEW
APPROVED
FINAL_RENDER_PENDING
FINAL_RENDERING
FINAL_RENDERED
DONE
FAILED
CANCELLED
```

Asset state:

```
UPLOADED
ANALYZE_PENDING
ANALYZED
PROXY_PENDING
PROXY_READY
ALPHA_PENDING
ALPHA_READY
READY
FAILED
```

15. Green Screen Compositing Requirements

System must support:

```
green screen keying
blue screen keying
custom key color
spill removal
```


edge feather
alpha matte output
foreground RGBA cache
scale / position
shadow
color matching
background resizing / crop

Important shooting standard:

Shoot foreground in 4K if possible
Subject 1.5-2 meters away from green screen
Even green screen lighting
No green objects if using green screen
If product is green, use blue screen
Lock exposure / focus / white balance
Use tripod
Leave space around subject for virtual camera crop

16. Recipe Scoring

Recipe score should consider:

product match
mood match
platform match
background/foreground fit
voice/music fit
script/CTA fit
layout safe area
motion compatibility
quality score
duplicate risk

Example score:

Recipe Score =
Product Match 20%
Mood Match 15%
Platform Match 10%
Visual Fit 15%

Audio Fit 15%
Script Fit 10%
Layout Fit 10%
Risk Penalty 5%

17. Duplicate / Repetition Control

The system should not be designed to evade platform detection.
It should be designed to avoid low-quality repetitive content.

Checks:

visual similarity
audio similarity
script similarity
caption similarity
template similarity
CTA similarity
metadata similarity
asset reuse frequency

Decision:

`duplicate_risk < 0.45 = OK`
`0.45-0.65 = Review`
`0.65-0.80 = Revise`
`> 0.80 = Reject`

18. Compliance and Ethical Rules

System must not support deceptive practices.

Rules:

Do not create fake reviews
Do not create fake social proof
Do not claim false sales
Do not use unauthorized people/voices
Do not use misleading health/financial claims

Do not use copyrighted music without rights
Do not mass-post low-value repetitive clips

System goal:

Create many high-quality variations, not spam

19. Minimum Viable Product

MVP 0.1

Create product
Upload asset
Store asset in folder
Save metadata in SQLite
Tag asset
Preview asset

MVP 0.2

Manual recipe selection
Select foreground
Select background
Select voiceover
Select music
Create recipe

MVP 0.3

Green screen removal
Composite foreground over background
Render preview

MVP 0.4

```
Render final  
Save output  
Save report
```

MVP 0.5

```
Auto-generate recipes  
Preview batch  
Approve/reject  
Final render approved recipes
```

20. First Development Order

Recommended implementation order:

1. SQLite schema
2. Product CRUD
3. Asset upload
4. Metadata analyzer using FFmpeg
5. Thumbnail generator
6. Tag system
7. Asset library UI
8. Manual recipe builder
9. Preview render pipeline
10. Green screen compositor
11. Audio mixer
12. Render job queue
13. Orchestrator
14. Worker system
15. Quality gate
16. Batch recipe generation
17. Final render
18. Output report

21. Definition of Done

A module is done when:

```
มีเอกสารอธิบาย
มี input/output ชัดเจน
มี error handling
มี log
มี test case
มีตัวอย่างไฟล์ใช้งาน
บันทึกสถานะลง SQLite
UI ใช้งานได้
ระบบ recover จาก error ได้
```

22. Critical Design Principles

```
Document first
Workflow before code
Preview before final
Cache before recompute
Tag before random
Quality before quantity
Orchestrator before worker chaos
SQLite as source of truth
Folder stores files, database stores meaning
Human review before publishing
```

23. Final Summary

This project is a workflow-driven AI video production system.

It consists of:

```
Product Robot
Media Library
Tag System
Recipe Engine
Green Screen Compositor
```

```
Audio Engine
Template Engine
Quality Gate
Orchestrator
Worker Queue
SQLite Database
Preview/Final Render Pipeline
```

The system should enable users to produce many product ad video variations from structured media ingredients while maintaining quality, traceability, performance, and compliance.

The correct philosophy:

- Do not build a random video generator.
- Build a controlled creative production factory.